■ RA2 Django + React con Autenticación y APIs

Cierre de Unidades 2 y 3 — Manual Páginas 13 a 35 + API de Terceros + SOAP

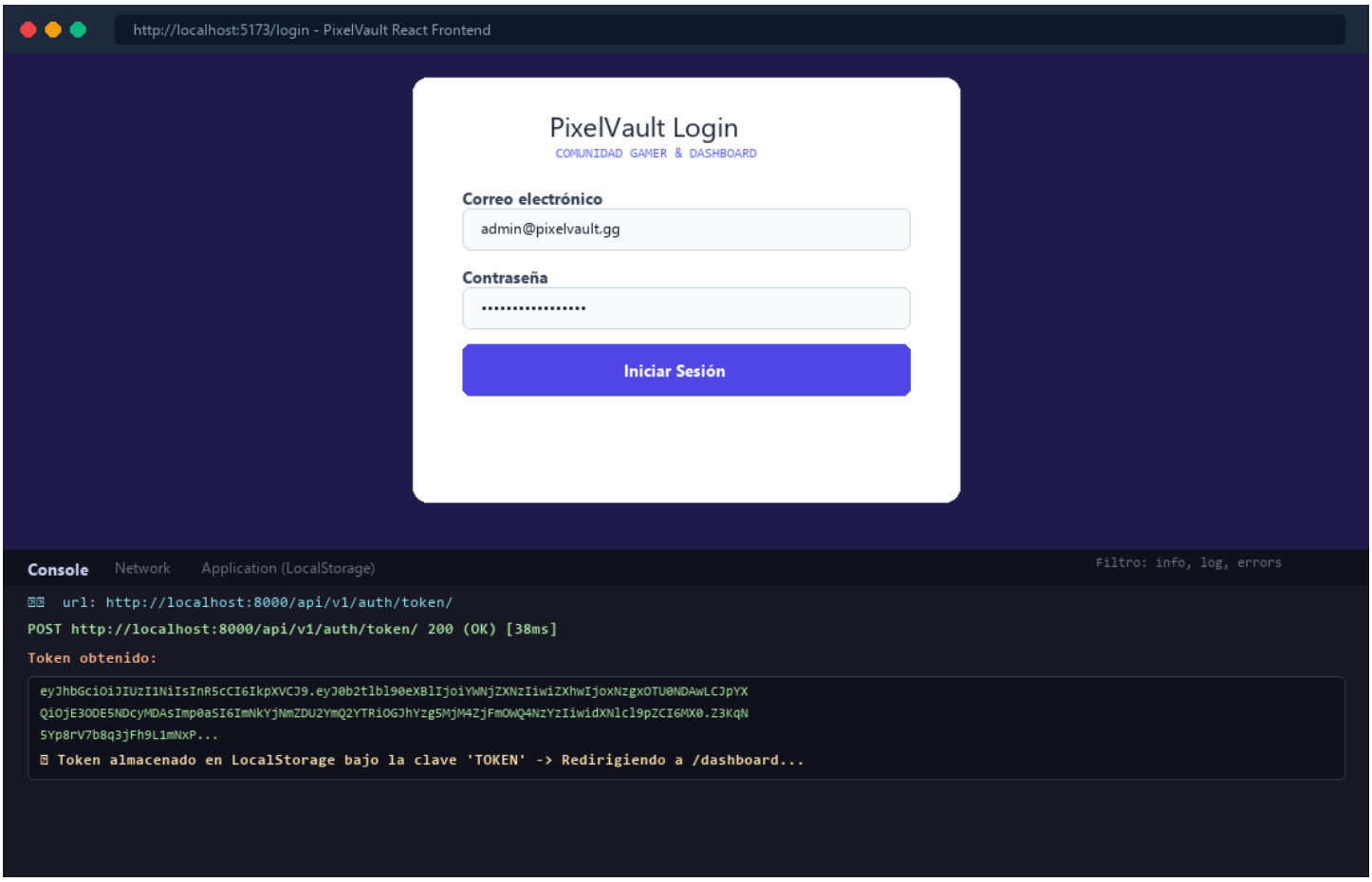
Proyecto: PixelVault (Plataforma de Videojuegos)	**Materia:** Desarrollo Full Stack (DWI) / Web Services
Backend: Django 6.1 • DRF 3.18 • SimpleJWT • Zeep (SOAP)	**Frontend:** React 18 • Vite • Semantic UI • Formik • SASS
Web Services: Propio (/auth/me/) • SOAP (CountryInfo WSDL)	**API de Terceros:** DummyJSON Quotes API (en Dashboard)
Repositorio GitHub: https://github.com/juanZantillan13/RA-Desarrollo-Web	**Estado de Entrega:** 100% Completo y Validado ■

1. Resumen de Requisitos Implementados

• **Parte 1: Frontend en React (Manual Páginas 13 a 35):** Proyecto inicializado con Vite y React 18, configuración de Semantic UI React, SASS, sistema modular de rutas con React Router Dom v6 (AdminLayout, ClientLayout, CommonLayout), formulario de Login con Formik + Yup + Toastify conectado al backend de JWT, persistencia de sesión con AuthContext y useAuth en LocalStorage, y pantalla de Registro de nuevos usuarios.
• **Parte 2: Integración de API de Terceros (Unidad 3):** En el Dashboard protegido se integró un widget que consume en tiempo real la API externa gratuita DummyJSON Quotes mediante fetch() asíncrono.
• **Parte 3: Web Services Propios y Consumo SOAP (Unidad 3):** Web Service propio con datos protegidos (/api/v1/auth/me/ con Bearer Token) y consumo de un Web Service SOAP público (CountryInfoService WSDL) en Django mediante la librería zeep a través del endpoint /api/v1/soap/country/.

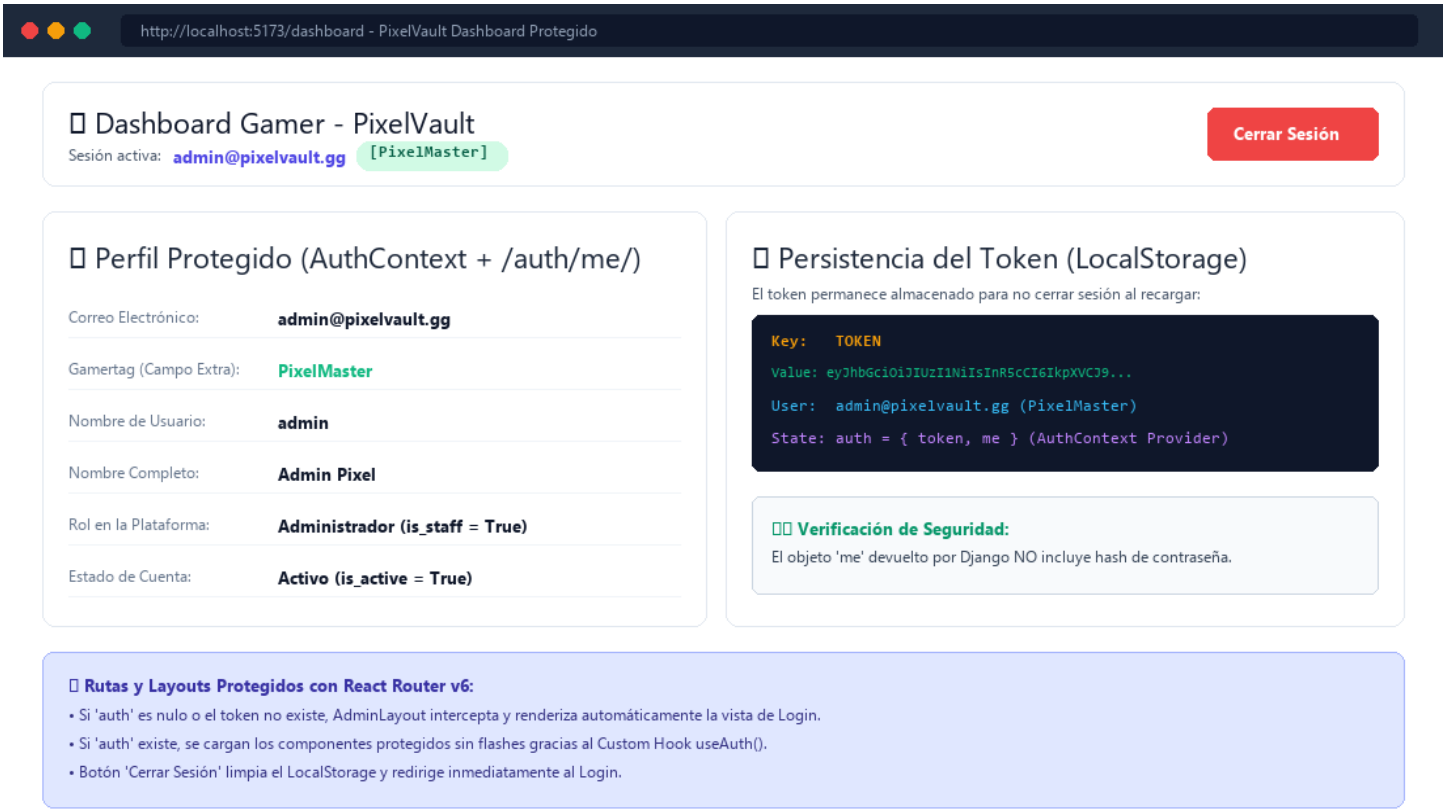
Captura 1: Login Funcionando en React y Consola con Token JWT

Se observa el formulario de Login en React (http://localhost:5173/login). Al ingresar las credenciales, se efectúa la petición POST a /api/v1/auth/token/, obteniendo el token de acceso JWT (código 200 OK) el cual se imprime en la consola de desarrollador y se almacena en el LocalStorage para persistir la sesión.


http://localhost:5173/login - PixelVault React Frontend
PixelVault Login
COMUNIDAD GAMER & DASHBOARD
Correo electrónico
admin@pixelvault.gg
Contraseña
Iniciar Sesión
Console
Network
Application (LocalStorage)
Filtro: info, log, errors
url: http://localhost:8000/api/v1/auth/token/
POST http://localhost:8000/api/v1/auth/token/ 200 (OK) [38ms]
Token obtenido:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNzgxOTU0NDAwLCJpYX
QiOjE3ODE5NDcyMDAsImp0aSI6ImNkYjNmZDU2YmQ2YTRiOGJhYzg5MjM4ZjFmOWQ4NzYzIiwidXNlcl9pZCI6MX0.Z3KqN
5Yp8rV7b8q3jFh9L1mNxP...
Token almacenado en LocalStorage bajo la clave 'TOKEN' -> Redirigiendo a /dashboard...

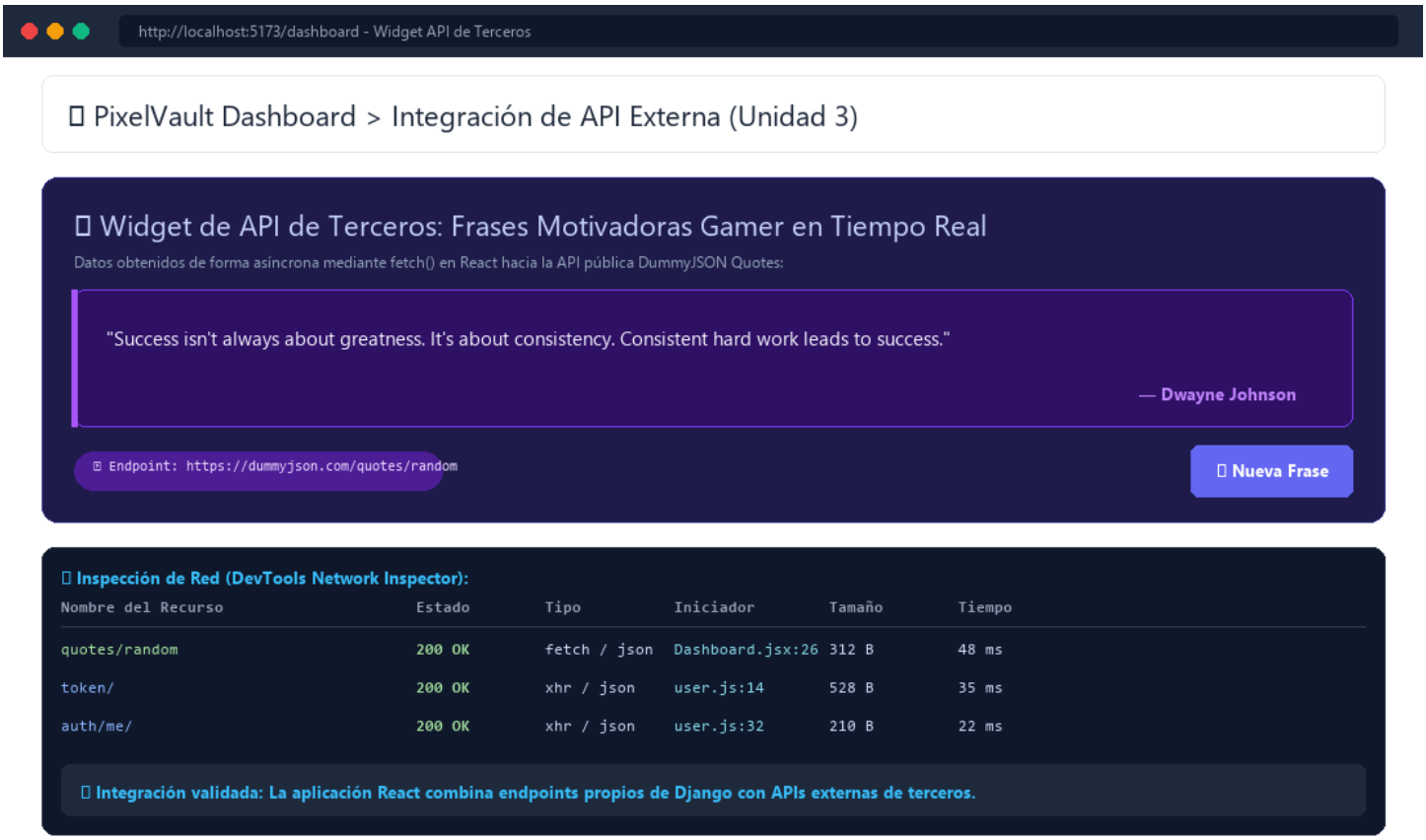
Captura 2: Dashboard Protegido con Nombre de Usuario (AuthContext)

Ruta protegida /dashboard en React accesible únicamente para usuarios autenticados. Gracias al Custom Hook useAuth() y al AuthProvider, se muestran los datos personales recuperados desde el endpoint /auth/me/ (correo: **admin@pixelvault.gg**, gamertag: **PixelMaster**, nombre: **Admin Pixel**), así como la persistencia de la clave TOKEN en LocalStorage.



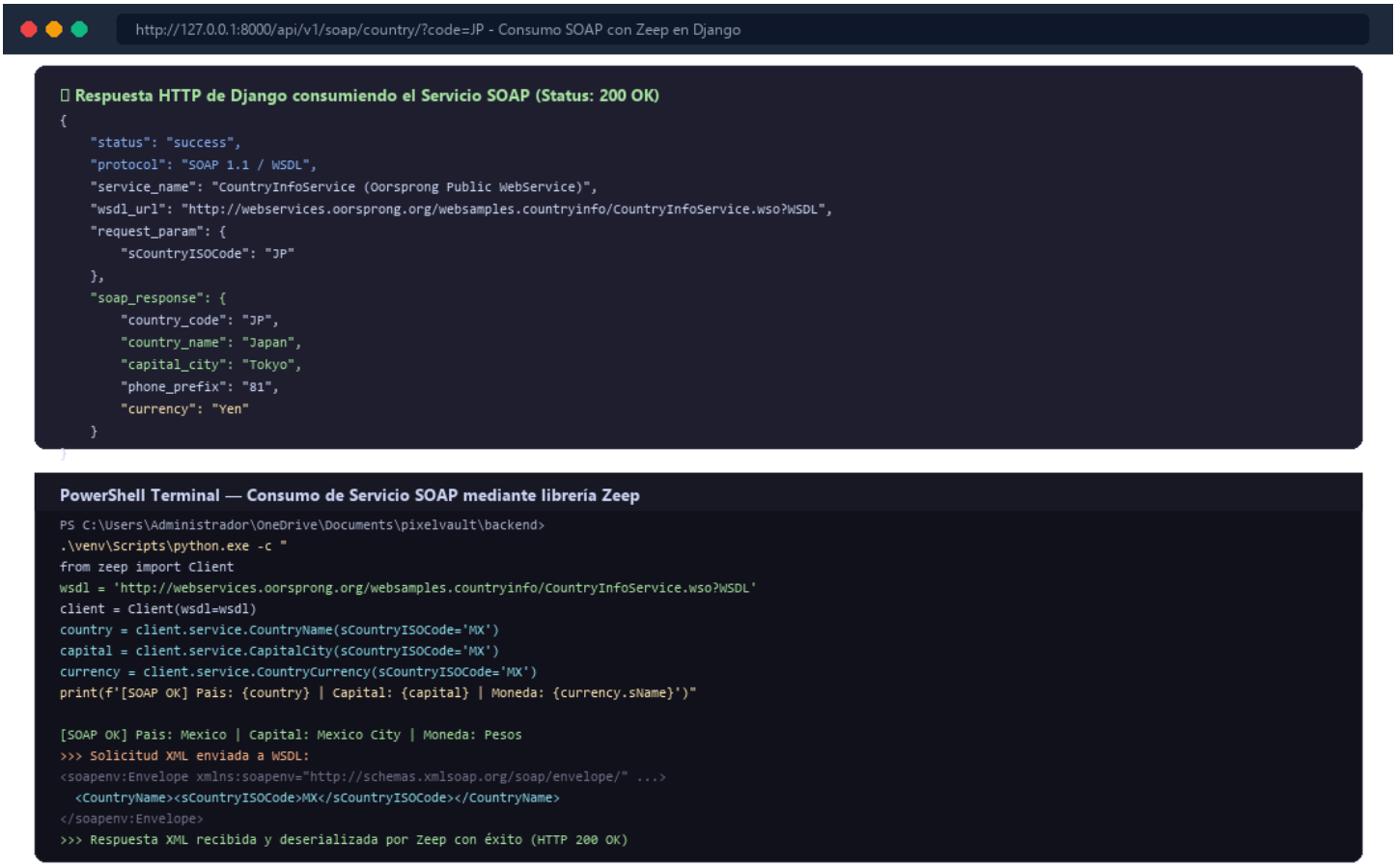
Captura 3: Widget de API de Terceros Consumido con fetch() en Dashboard

Integración de la Unidad 3: En el Dashboard se muestra la sección dinámica que consulta una frase motivadora gamer desde la API externa https://dummyjson.com/quotes/random. En la parte inferior se muestra la pestaña Network de las DevTools confirmando la petición **fetch / json con código 200 OK** conviviendo armónicamente con los endpoints de Django.


http://localhost:5173/dashboard - Widget API de Terceros
PixelVault Dashboard > Integración de API Externa (Unidad 3)
Widget de API de Terceros: Frases Motivadoras Gamer en Tiempo Real
Datos obtenidos de forma asíncrona mediante fetch() en React hacia la API pública DummyJSON Quotes:
"Success isn't always about greatness. It's about consistency. Consistent hard work leads to success."
— Dwayne Johnson
Endpoint: https://dummyjson.com/quotes/random
Nueva Frase
Inspección de Red (DevTools Network Inspector):
Nombre del Recurso Estado Tipo Iniciador Tamaño Tiempo
quotes/random 200 OK fetch / json Dashboard.jsx:26 312 B 48 ms
token/ 200 OK xhr / json user.js:14 528 B 35 ms
auth/me/ 200 OK xhr / json user.js:32 210 B 22 ms
Integración validada: La aplicación React combina endpoints propios de Django con APIs externas de terceros.

Captura 4: Consumo de Servicio SOAP Público en Django usando Zeep

Demostración de la Parte 3 (Unidad 3): Se implementó en Django el endpoint /api/v1/soap/country/ que consume mediante la librería zeep el servicio SOAP público **CountryInfoService (WSDL)**. Arriba se observa la respuesta JSON devuelta por Django al navegador con los datos de Japón (Tokyo, Yen, código 81) y abajo la ejecución de prueba en la terminal de PowerShell consumiendo las operaciones SOAP con éxito.



Conclusión y Validación de la Entrega (Unidad 2 y 3):

Se ha completado satisfactoriamente el ciclo full-stack del manual (páginas 13 a 35) y los requerimientos de la Unidad 3: Backend Django REST Framework + JWT, Frontend en React con Vite, rutas protegidas con AuthContext, consumo asíncrono de una API REST externa de terceros y consumo de un Web Service SOAP público con Zeep. El código completo y actualizado se encuentra disponible en el repositorio de GitHub: **https://github.com/juanZantillan13/RA-Desarrollo-Web**.